

USE CLAUDE CODE

Choose a permission mode

Control whether Claude asks before editing files or running commands. Cycle modes with Shift+Tab in the CLI or use the mode selector in VS Code, Desktop, and claude.ai.

When Claude wants to edit a file, run a shell command, or make a network request, it pauses and asks you to approve the action. Permission modes control how often that pause happens. The mode you pick shapes the flow of a session: default mode has you review each action as it comes, while looser modes let Claude work in longer uninterrupted stretches and report back when done. Pick more oversight for sensitive work, or fewer interruptions when you trust the direction.

Available modes

Each mode makes a different tradeoff between convenience and oversight. The table below shows what Claude can do without a permission prompt in each mode.

Mode	What runs without asking	Best for
default	Reads only	Getting started, sensitive work
<u>acceptEdits</u>	Reads, file edits, and common filesystem commands (<code>mkdir</code> , <code>touch</code> , <code>mv</code> , <code>cp</code> , etc.)	Iterating on code you're reviewing
<u>plan</u>	Reads only	Exploring a codebase before changing it
<u>auto</u>	Everything, with background safety checks	Long tasks, reducing prompt fatigue
<u>dontAsk</u>	Only pre-approved tools	Locked-down CI and scripts
<u>bypassPermissions</u>	Everything	Isolated containers and VMs only

In every mode except `bypassPermissions` , writes to protected paths are never auto-approved, guarding repository state and Claude's own configuration against accidental corruption.

Modes set the baseline. Layer permission rules on top to pre-approve or block specific tools. Deny

rules and explicit ask rules apply in every mode, including `bypassPermissions`. Allow rules have no effect in that mode because everything else is already approved.

Switch permission modes

You can switch modes mid-session, at startup, or as a persistent default. The mode is set through these controls, not by asking Claude in chat. Select your interface below to see how to change it.

CLI

During a session: press `Shift+Tab` to cycle `default` → `acceptEdits` → `plan`. The current mode appears in the status bar. Not every mode is in the default cycle:

- `auto`: appears when your account meets the [auto mode requirements](#); cycling to auto shows an opt-in prompt until you accept it, or select **No, don't ask again** to remove auto from the cycle
- `bypassPermissions`: appears after you start with `--permission-mode bypassPermissions`, `-dangerously-skip-permissions`, or `--allow-dangerously-skip-permissions`; the `--allow-` variant adds the mode to the cycle without activating it
- `dontAsk`: never appears in the cycle; set it with `--permission-mode dontAsk`

Enabled optional modes slot in after `plan`, with `bypassPermissions` first and `auto` last. If you have both enabled, you will cycle through `bypassPermissions` on the way to `auto`. **At startup:** pass the mode as a flag.

```
claude --permission-mode plan
```

As a default: set `defaultMode` in [settings](#).

```
{
  "permissions": {
    "defaultMode": "acceptEdits"
  }
}
```

The same `--permission-mode` flag works with `-p` for [non-interactive runs](#).

VS Code

During a session: click the mode indicator at the bottom of the prompt box.**As a default:** set `claudeCode.initialPermissionMode` in VS Code settings, or use the Claude Code extension settings panel.The mode indicator shows these labels, mapped to the mode each one applies:

UI label	Mode
Ask before edits	default
Edit automatically	acceptEdits
Plan mode	plan
Auto mode	auto
Bypass permissions	bypassPermissions

Auto mode appears in the mode indicator when your account meets every requirement listed in the [auto mode section](#). The `claudeCode.initialPermissionMode` setting does not accept `auto`. To start in auto mode by default, set `defaultMode` in your [user settings](#) instead. Claude Code ignores `defaultMode: "auto"` in project and local settings.Bypass permissions requires the **Allow dangerously skip permissions** toggle in the extension settings before it appears in the mode indicator.See the [VS Code guide](#) for extension-specific details.

JetBrains

The JetBrains plugin runs Claude Code in the IDE terminal, so switching modes works the same as in the CLI: press `Shift+Tab` to cycle, or pass `--permission-mode` when launching.

Desktop

Use the mode selector next to the send button. Auto and Bypass permissions appear only after you enable them in Desktop settings. See the [Desktop guide](#).

Web and mobile

Use the mode dropdown next to the prompt box on [claude.ai/code](#) or in the mobile app. Permission

prompts appear in `claude.ai` for approval. Which modes appear depends on where the session runs:

- **Cloud sessions** on **Claude Code on the web**: Accept edits, Plan mode, and Auto mode. Accept edits corresponds to `default` mode: the cloud environment pre-approves file edits regardless of mode, so the dropdown shows Accept edits instead of Ask permissions. `defaultMode: "acceptEdits"` from settings is still honored. Auto mode appears only when your organization allows it and the selected model supports it. Bypass permissions is not available.
- **Remote Control sessions** on your local machine: Ask permissions, Auto accept edits, and Plan mode. Auto and Bypass permissions are not available.

For Remote Control, you can also set the starting mode when launching the host:

```
claude remote-control --permission-mode acceptEdits
```

Auto-approve file edits with `acceptEdits` mode

`acceptEdits` mode lets Claude create and edit files in your working directory without prompting. The status bar shows `▶▶ accept edits on` while this mode is active.

In addition to file edits, `acceptEdits` mode auto-approves common filesystem Bash commands: `mkdir`, `touch`, `rm`, `rmdir`, `mv`, `cp`, and `sed`. These commands are also auto-approved when prefixed with safe environment variables such as `LANG=C` or `NO_COLOR=1`, or process wrappers such as `timeout`, `nice`, or `nohup`. Like file edits, auto-approval applies only to paths inside your working directory or `additionalDirectories`. Paths outside that scope, writes to **protected paths**, and all other Bash commands still prompt.

When the **PowerShell tool** is enabled, `acceptEdits` mode also auto-approves `Set-Content`, `Add-Content`, `Clear-Content`, and `Remove-Item` on in-scope paths, along with their common aliases. The same scope and protected-path rules apply.

Use `acceptEdits` when you want to review changes in your editor or via `git diff` after the fact rather than approving each edit inline. Press `Shift+Tab` once from default mode to enter it, or start with it directly:

```
claude --permission-mode acceptEdits
```

Analyze before you edit with plan mode

Plan mode tells Claude to research and propose changes without making them. Claude reads files, runs shell commands to explore, and writes a plan, but does not edit your source. Permission prompts still apply the same as default mode.

Enter plan mode by pressing `Shift+Tab` or prefixing a single prompt with `/plan`. You can also start in plan mode from the CLI:

```
claude --permission-mode plan
```

Press `Shift+Tab` again to leave plan mode without approving a plan.

Review and approve a plan

When the plan is ready, Claude presents it and asks how to proceed. From that prompt you can:

- Approve and start in auto mode
- Approve and accept edits
- Approve and review each edit manually
- Keep planning with feedback
- Refine with [Ultraplan](#) for browser-based review

Approving a plan exits plan mode and switches the session to the permission mode each approve option describes, so Claude starts editing. To plan again, cycle back to plan mode with `Shift+Tab`, or prefix your next prompt with `/plan`.

Press `Ctrl+G` to open the proposed plan in your default text editor and edit it directly before Claude proceeds. When `showClearContextOnPlanAccept` is enabled, each approve option also offers to clear the planning context first.

Accepting a plan also names the session from the plan content automatically, unless you've already set a name with `--name` or `/rename`.

Set plan mode as the default

To make plan mode the default for a project, set `defaultMode` in `.claude/settings.json`:

```
{
  "permissions": {
    "defaultMode": "plan"
  }
}
```

Eliminate permission prompts with auto mode

ⓘ Auto mode requires Claude Code v2.1.83 or later.

Auto mode lets Claude execute without routine permission prompts. A separate classifier model reviews actions before they run, blocking anything that escalates beyond your request, targets unrecognized infrastructure, or appears driven by hostile content Claude read. Explicit ask rules still force a prompt.

Auto mode also nudges Claude to keep working without stopping for clarifying questions, though Claude still asks when your prompt or a skill explicitly relies on it. For stronger autonomous behavior while keeping permission prompts, set the Proactive output style instead.

⚠ Auto mode is a research preview. It reduces permission prompts but does not guarantee safety. Use it for tasks where you trust the general direction, not as a replacement for review on sensitive operations.

Auto mode is available only when your account meets all of these requirements:

- **Plan:** All plans.
- **Owner:** on Team and Enterprise, an Owner must enable it in Claude Code admin settings before users can turn it on. Administrators can also lock it off by setting `permissions.disableAutoMode` to `"disable"` in managed settings.
- **Model:** on the Anthropic API, Claude Opus 4.6 or later, or Sonnet 4.6. On Amazon Bedrock, Google Cloud Vertex AI, and Microsoft Foundry, only Claude Opus 4.7 and Opus 4.8. Older models, including Sonnet 4.5, Opus 4.5, Haiku, and claude-3 models, are not supported on any provider.
- **Provider:** available by default on the Anthropic API. On Amazon Bedrock, Google Cloud Vertex AI, and Microsoft Foundry, auto mode is off until you set `CLAUDE_CODE_ENABLE_AUTO_MODE`.

If Claude Code reports auto mode as unavailable, one of these requirements is unmet; this is not a transient outage. A separate message that names a model and says auto mode “cannot determine

the safety” of an action is a transient classifier outage; see the [error reference](#).

If you set `defaultMode: "auto"` in [settings](#) and the session starts in `default` mode with no error, the setting is likely in `.claude/settings.json` or `.claude/settings.local.json`. Claude Code v2.1.142 and later ignore `auto` from those files so a repository cannot grant itself auto mode. Move it to `~/.claude/settings.json`.

Enable auto mode on Bedrock, Vertex AI, or Foundry

On [Amazon Bedrock](#), [Google Cloud Vertex AI](#), and [Microsoft Foundry](#), auto mode does not appear in the `Shift+Tab` cycle until `CLAUDE_CODE_ENABLE_AUTO_MODE` is set to `1`. The variable works in Claude Code v2.1.158 and later. Only Claude Opus 4.7 and Opus 4.8 are supported on these providers.

To enable it for one developer, add the variable to the `env` block in `~/.claude/settings.json`:

```
{
  "env": {
    "CLAUDE_CODE_ENABLE_AUTO_MODE": "1"
  }
}
```

To enable it for your organization, add the same `env` block to [managed settings](#).

Once the variable is set, auto mode appears in the `Shift+Tab` cycle for every session. To make it the default starting mode, also set `"permissions": {"defaultMode": "auto"}` in user or managed settings. On these providers, Claude Code ignores `defaultMode: "auto"` unless `CLAUDE_CODE_ENABLE_AUTO_MODE` is also set.

To prevent developers from enabling auto mode, set `disableAutoMode` to `"disable"` in managed settings. This overrides the enable variable.

If you connect through an [LLM gateway](#) configured with `ANTHROPIC_BASE_URL`, auto mode may already be reachable without the enable variable, because the gateway routes requests through the Anthropic API. The `disableAutoMode` setting applies the same way in that configuration.

What the classifier blocks by default

The classifier trusts your working directory and your repo’s configured remotes. Everything else is treated as external until you [configure trusted infrastructure](#).

Blocked by default:

- Downloading and executing code, like `curl` | `bash`
- Sending sensitive data to external endpoints
- Production deploys and migrations
- Mass deletion on cloud storage
- Granting IAM or repo permissions
- Modifying shared infrastructure
- Irreversibly destroying files that existed before the session
- Force push, or pushing directly to `main`
- `git reset --hard` , `git checkout -- .` , `git restore .` , `git clean -fd` , `git stash drop` , or `git stash clear` , which the classifier presumes would discard uncommitted changes
- `git commit --amend` when the commit at HEAD was not created in this session
- `terraform destroy` , `pulumi destroy` , `cdk destroy` , or `terragrunt destroy` , and applying a plan that destroys resources

Allowed by default:

- Local file operations in your working directory
- Installing dependencies declared in your lock files or manifests
- Reading `.env` and sending credentials to their matching API
- Read-only HTTP requests
- Pushing to the branch you started on or one Claude created

Sandbox network access requests are routed through the classifier rather than allowed by default. Run `claude auto-mode defaults` to see the full rule lists. If routine actions get blocked, an administrator can add trusted repos, buckets, and services via the `autoMode.environment` setting: see [Configure auto mode](#).

Boundaries you state in conversation

The classifier treats boundaries you state in the conversation as a block signal. If you tell Claude “don’t push” or “wait until I review before deploying”, the classifier blocks matching actions even when the default rules would allow them. A boundary stays in force until you lift it in a later message.

Claude's own judgment that a condition was met does not lift it.

Boundaries are not stored as rules. The classifier re-reads them from the transcript on each check, so a boundary can be lost if **context compaction** removes the message that stated it. For a hard guarantee, add a **deny rule** instead.

When auto mode falls back

Each denied action shows a notification and appears in `/permissions` under the Recently denied tab, where you can press `r` to retry it with a manual approval.

If the classifier blocks an action 3 times in a row or 20 times total, auto mode pauses and Claude Code resumes prompting. Approving the prompted action resumes auto mode. These thresholds are not configurable. Any allowed action resets the consecutive counter, while the total counter persists for the session and resets only when its own limit triggers a fallback.

In **non-interactive mode** with the `-p` flag, repeated blocks abort the session since there is no user to prompt.

Repeated blocks usually mean the classifier is missing context about your infrastructure. Use `/feedback` to report false positives, or have an administrator **configure trusted infrastructure**.

- How the classifier evaluates actions
- How auto mode handles subagents
- Cost and latency

Allow only pre-approved tools with dontAsk mode

`dontAsk` mode auto-denies every tool call that would otherwise prompt. Only actions matching your `permissions.allow` rules and **read-only Bash commands** can execute; explicit **ask rules** are denied rather than prompting. This makes the mode fully non-interactive for CI pipelines or restricted environments where you pre-define exactly what Claude may do. Cloud sessions on **Claude Code on the web** ignore `defaultMode: "dontAsk"`; see **bypassPermissions** for details.

Set it at startup with the flag:

```
claude --permission-mode dontAsk
```

Skip all checks with `bypassPermissions` mode

`bypassPermissions` mode disables permission prompts and safety checks so tool calls execute immediately. As of v2.1.126 this includes writes to protected paths, which earlier versions still prompted for. Explicit ask rules still force a prompt in this mode, and removals targeting the filesystem root or home directory, such as `rm -rf /` and `rm -rf ~`, still prompt as a circuit breaker against model error. Only use this mode in isolated environments like containers, VMs, or dev containers without internet access, where Claude Code cannot damage your host system.

You cannot enter `bypassPermissions` from a session that was started without one of the enabling flags; restart with one to enable it:

```
claude --permission-mode bypassPermissions
```

The `--dangerously-skip-permissions` flag is equivalent.

On Linux and macOS, Claude Code refuses to start in this mode when running as root or under `sudo` :

```
--dangerously-skip-permissions cannot be used with root/sudo
privileges for security reasons
```

The check is skipped automatically inside a recognized sandbox. To run autonomously in a container, use the dev container configuration, which runs Claude Code as a non-root user.

Claude Code on the web does not honor `defaultMode: "bypassPermissions"` or `"dontAsk"` from your settings files, so a repository's checked-in settings cannot start a cloud session in `bypassPermissions` mode. The setting is ignored silently and the session starts in the mode shown in the mode dropdown instead. See Switch permission modes for which modes cloud sessions offer.

⚠ `bypassPermissions` offers no protection against prompt injection or unintended actions. For background safety checks with far fewer permission prompts, use auto mode instead. Administrators can block this mode by setting `permissions.disableBypassPermissionsMode` to `"disable"` in managed settings.

Protected paths

Writes to a small set of paths are never auto-approved, in every mode except `bypassPermissions` . This prevents accidental corruption of repository state and Claude’s own configuration.

Mode	Protected-path writes
<code>default</code> , <code>acceptEdits</code> , <code>plan</code>	Prompted
<code>auto</code>	Routed to the classifier
<code>dontAsk</code>	Denied
<code>bypassPermissions</code>	Allowed

`permissions.allow` rules in settings files do not pre-approve protected-path writes. The safety check runs before Claude Code evaluates allow rules from settings, so an entry such as `Edit(.claude/**)` in `~/.claude/settings.json` or `.claude/settings.json` does not change the per-mode outcome in the table above. In modes that prompt, the prompt for a `.claude/` write offers **Yes, and allow Claude to edit its own settings for this session**, which approves later `.claude/` writes in that session without prompting again.

Protected directories:

- `.git`
- `.config/git`
- `.vscode`
- `.idea`
- `.husky`
- `.cargo`
- `.devcontainer`
- `.yarn`
- `.mvn`
- `.claude` , except for `.claude/worktrees` where Claude stores its own git worktrees

Protected files:

- `.gitconfig` , `.gitmodules`
- `.bashrc` , `.bash_profile` , `.bash_login` , `.bash_aliases` , `.bash_logout` , `.zshrc` ,

- `.zprofile` , `.zshenv` , `.zlogin` , `.zlogout` , `.profile` , `.envrc`
- `.npmrc` , `.yarnrc` , `.yarnrc.yml` , `.pnp.cjs` , `.pnp.loader.mjs` , `.pnpmfile.cjs` , `bunfig.toml` , `.bunfig.toml`
- `.bazelrc` , `.bazelversion` , `.bazeliskrc`
- `.pre-commit-config.yaml` , `lefthook.yml` , `lefthook.yaml` , `.lefthook.yml` , `.lefthook.yaml`
- `gradle-wrapper.properties` , `maven-wrapper.properties`
- `.devcontainer.json`
- `.ripgreprc` , `pyrightconfig.json`
- `.mcp.json` , `.claude.json`

See also

- **Permissions**: allow, ask, and deny rules; managed policies
- **Configure auto mode**: tell the classifier which infrastructure your organization trusts
- **Hooks**: custom permission logic via `PreToolUse` and `PermissionRequest` hooks
- **Ultraplan**: run plan mode in a Claude Code on the web session with browser-based review
- **Security**: safeguards and best practices
- **Sandboxing**: filesystem and network isolation for Bash commands
- **Non-interactive mode**: run Claude Code with the `-p` flag